Homework 4 Report

1. Approximate KL

In a short paragraph, explain why the estimator is a valid sampled-token estimator for the KL term used in this assignment, and why computing the exact full-vocabulary KL at every token position would be much more expensive in both compute and memory.

The KL divergence between the current policy (π_Θ) and the reference policy (π_{ref}) is approximated using the sampled token estimator mentioned in the hw4 pdf ($e^{\Delta}-\Delta-1$). This estimator is valid because its expectation under a ~ π_Θ equals to the true KL divergence.

E[estimator] = KL($\pi_\Theta(\cdot|s)||\pi_{ref}(\cdot|s)$)

The formulation is always non-negative and behaves like a true divergence at the per-token level. Computing the exact KL divergence requires summing over the entire vocab at every token position, which is computationally expensive in time and memory. This sampled estimator only requires us evaluating the log prob of the sampled tokens, which is more efficient.

2. Implementation

Briefly describe the order in which you implemented the TODOs and one bug or confusion point that you had to resolve.

I followed the order that is recommended in the pdf.

1. First, I complete the compute_per_token_logprobs, build_completion_mask, approx_kl_from_logprobs functions. These are pure tensor utilities with no dependencies on anything else, so they had to come first. Other downstream components call them.
2. iter_minibatches: needed before any update() could run.
3. compute_group_advantages, maybe_normalize_advantages: needed before building the RolloutBatch.advantages field passed into the algorithms.
4. Reinforce.update: simpler algorithm, no clipping or importance ratios, good to validate first.
5. GRPO.update: extends REINFORCE with the PPO clipped surrogate and cached old_logprobs.

The felt like the trickiest part was the off-by-one indexing in build_completion_mask. The per-token logprob tensor has shape [B, L-1], where index t scores token input_ids[:, t+1]. So the first completion token sits at input_ids[:, prompt_input_len], which corresponds to logprob index prompt_input_len - 1, but not prompt_input_len. The mask condition is therefore positions >= prompt_input_len - 1, not >= prompt_input_len. Although the code contained hints, fully

understanding and verifying this alignment took a significant amount of time. I had to carefully trace the indexing through several tensors and double-check shapes at each step.

3. GR-REINFORCE vs GRPO on Math

Compare the WandB curves for the math runs. How do the two methods differ over the first 200 iterations? Why is that comparison interesting given the way the provided commands were chosen?

Comparing the WandB curves for the math_hard task, GRPO shows faster improvement than GR-REINFORCE during the early stages of training.

In the first 200 iterations:

- eval/math_hard_test_subset_split_fraction_exact_match_using_boxed_answer_parser
    - GR-REINFORCE achieved a final evaluation accuracy of approximately 0.30078
    - GRPO achieved a higher evaluation accuracy of approximately 0.35156

The training time reward metrics (rollout/mean_total_reward_across_all_completions_in_batch_and_groups) fluctuate a lot, so there's no insight gained from that graph.

The key reason is probably due to GRPO reusing rollout data for multiple optimization steps via PPO-style updates (ppo_epochs=2), while GR-REINFORCE performs only a single on-policy update per rollout batch. GRPO is more sample efficient and makes more progress per collected batch. This comparison is interesting given the way the provided commands were chosen because both runs use the same rollout size and hyperparameters, other than the effect of sample reuse; it's a good ablation study.

4. GRPO Ablations on Format Copy

Summarize the extra GRPO runs you tried. Which hyperparameters mattered most? Which settings made learning worse, and why do you think they did?

Baseline:

```
uv run modal run --detach scripts/modal_train.py -- \
  --task format_copy \
  --algo grpo \
  --output_dir /vol/runs/modal_format_copy_grpo_default \
  --steps 51 \
  --batch_size 8 \
  --group_size 6 \
  --min_new_tokens 1 \
  --max_new_tokens 24 \
  --lr 3e-5 \
  --ppo_epochs 2 \
```

```
--minibatch_size 8 \
--grad_accum_steps 6 \
--clip_eps 0.2 \
--kl_coef 0.05 \
--max_grad_norm 0.5 \
--wandb_enabled \
--wandb_project llm-rl-hw4 \
--wandb_name format_copy_grpo_default \
--sample_markdown_log_interval 1 \
--sample_log_interval 10 \
--sample_log_n 6 \
--eval_interval 50 \
--save_interval 50 \
--warmup_steps 10
```

I conducted an ablation study on each hyperparameter stated below. Due to the limited amount of computation and not wanting to use my own budget, I didn't test each parameter until it degraded in some of the ablation. However, I still observed a clear effect of each.

PPO Epochs

I varied PPO epochs across values {1, 2 (baseline), 4, 6}.

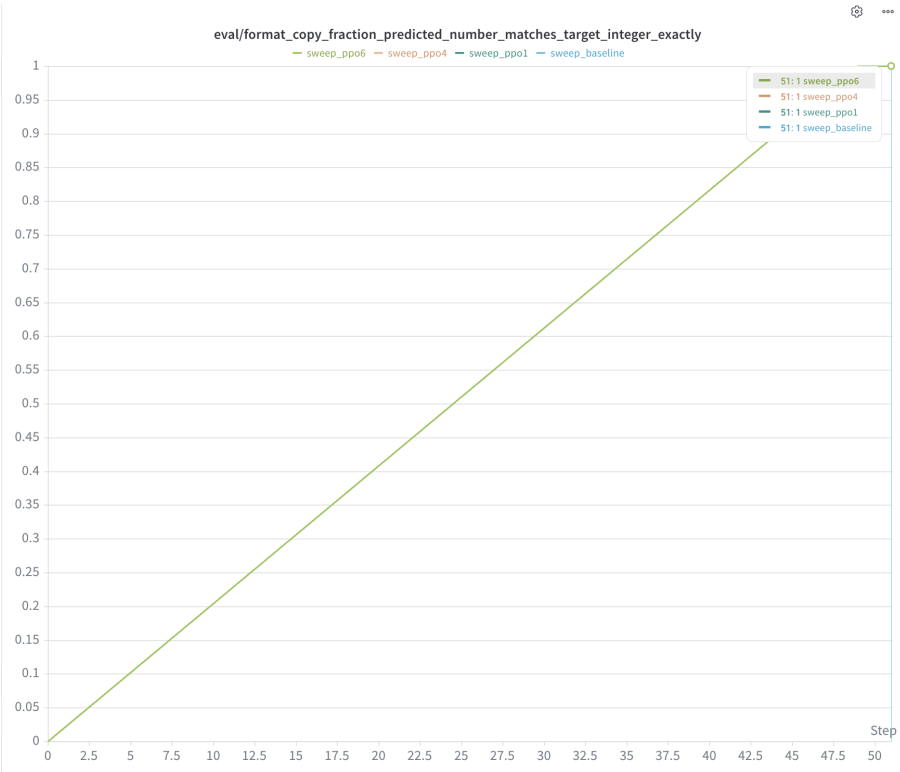


Figure 1: This graph shows evaluation metrics for across PPO epochs with everything else fixed. "sweep_ppoX" represents X epochs and sweep_baseline represents 2 epochs. All runs saturated and achieved near-perfect evaluation accuracy (1.0)
on the format_copy task.

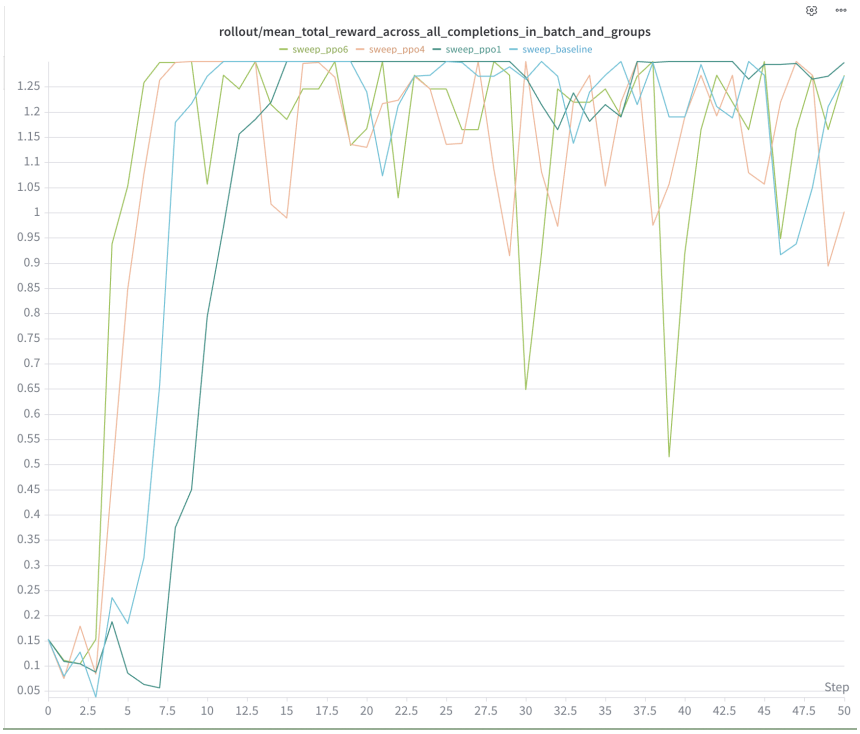


Figure 2: This graph shows training-time reward metric for across PPO epochs with everything else fixed. “sweep_ppoX” represents X epochs and sweep_baseline represents 2 epochs. All runs saturated quickly, however there’s a trend where higher PPO epochs reach saturation point faster.

This makes sense since lower PPO epochs means fewer updates were applied per rollout. Increasing PPO epochs to 4 improved sample efficiency, allowing faster convergence. At PPO epochs = 6, the training became slightly less stable towards later steps, which might be due to over-updating.

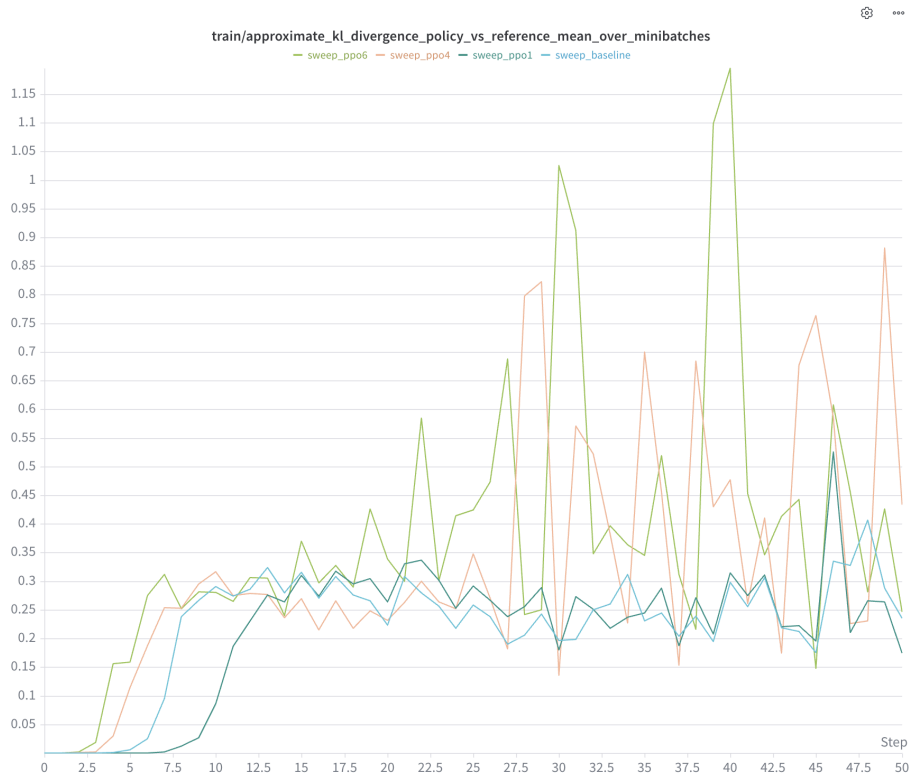


Figure 3: This graph shows KL divergence between the policy and the reference model. “sweep_ppoX” represents X epochs and sweep_baseline represents 2 epochs. When ppo_epochs=1, the KL remained low and stable. The baseline setting (ppo_epochs=2) maintained a moderate KL with stable behavior. Increasing PPO epochs to 4 and 6 resulted in higher and more variable KL values and spikes.

I analyzed the effect of PPO epochs on the KL divergence between the policy and the reference model. Increasing PPO epochs led to a clear and earlier increase in KL divergence, as the model performed more optimization steps on the same rollout data. This is probably due to reusing the same rollout data for multiple optimization passes will cause the model to deviate further from reference distribution and be more unstable in training.

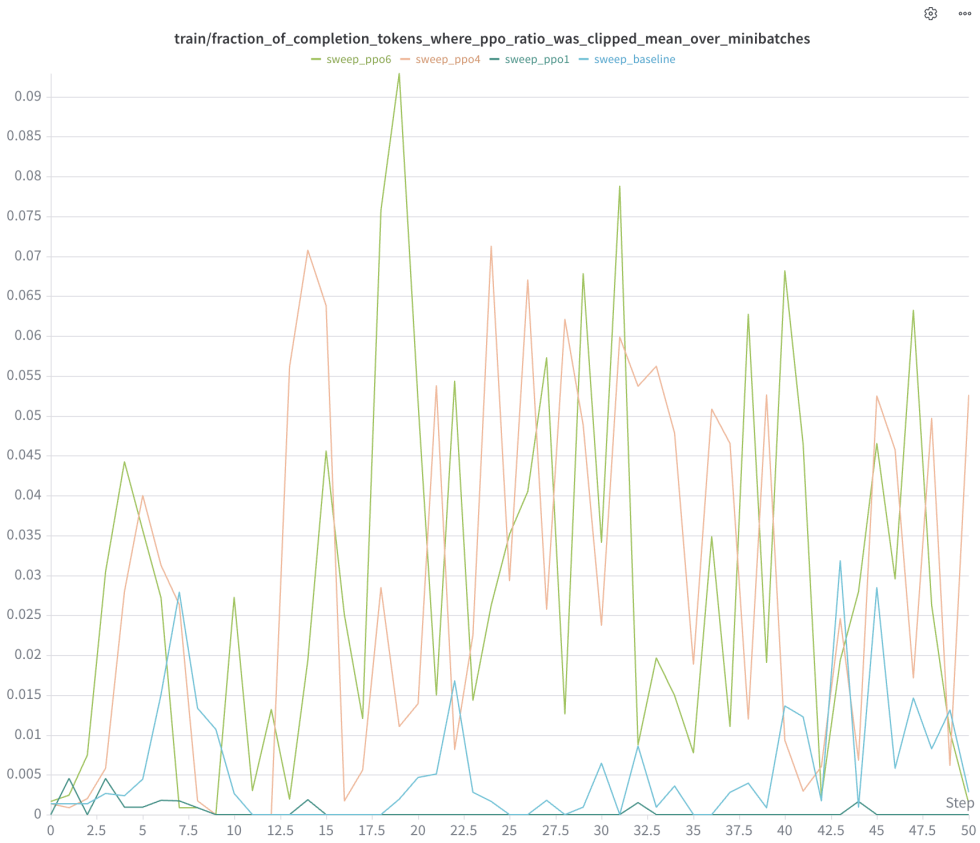


Figure 4: This graph shows the fraction of completion tokens where the PPO ratio was clipped. “sweep_ppoX” represents X epochs and sweep_baseline represents 2 epochs. When ppo_epochs=1, the clip fraction remained low and stable. The baseline setting (ppo_epochs=2) maintained a moderate clip fraction with stable behavior. Increasing PPO epochs to 4 and 6 resulted in higher and more variable clip fraction values and spikes.

As PPO epochs increase, the model performs more optimization steps on the same rollout data, which causes the importance ratio between the current policy and the old policy to deviate further from 1. This leads to more tokens exceeding the clipping threshold, increasing the fraction of clipped updates. With ppo_epochs=1, updates are small and rarely require clipping, resulting in low clip fraction. At moderate values (ppo_epochs=2), clipping occurs occasionally. However, at higher values (ppo_epochs=4 and 6), repeated updates push the policy too far, causing frequent clipping and large spikes in clip fraction. This shows over-updating, where PPO’s clipping mechanism limits unstable updates.

KL Coefficient

I varied KL coefficients across values {0.001, 0.005, 0.05 (baseline), 0.1, 0.5, 2.0, 5.0}.

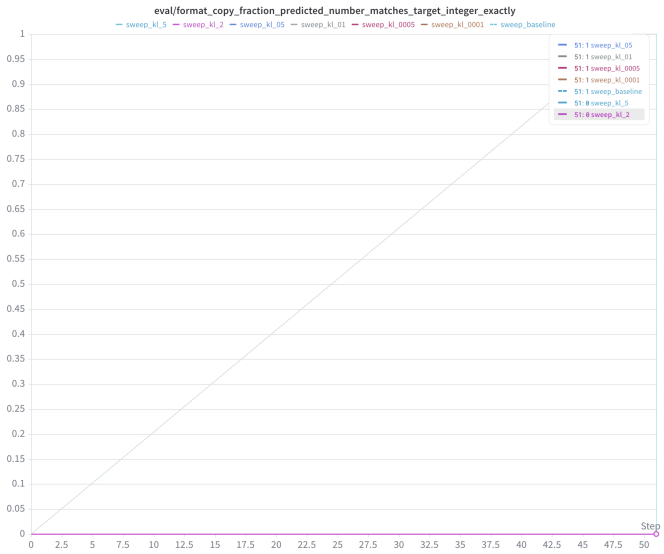


Figure 5: This graph shows evaluation metrics for across KL coefficients with everything else fixed. “sweep_kl_X” represents KL coefficient X and sweep_baseline represents 0.05. KL coefficient of 2.0 and 5.0 did not reach evaluation accuracy of 1.0 on the format_copy task.

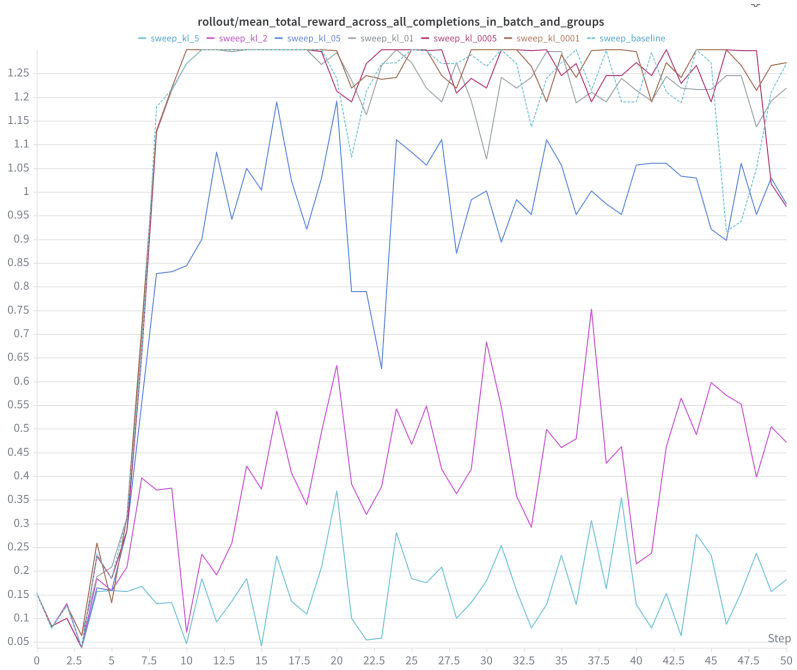


Figure 6: This graph shows training-time reward metric for across KL coefficients with everything else fixed. “sweep_kl_X” represents KL coefficient X and sweep_baseline represents 0.05. KL coefficient of 0.001, 0.005, 0.05, 0.1 reached near maximum reward around 10 steps. Larger KL coefficients (0.5, 2.0, 5.0) yield progressively lower rewards. All rewards increase and saturate at similar timestep.

KL coefficient controls how strongly the policy is constrained to stay close to the reference model. Smaller KL values allow the model to update more freely, leading to faster learning and higher rewards. As the KL coefficient increases, the penalty for deviating from the reference policy becomes stronger, limiting the magnitude of updates. This results in slower improvement and lower achievable reward in limited timestep. At very large KL values (2.0 and 5.0), the constraint is so strong that the model is prevented from learning the task fully, which explains why these runs fail to reach perfect evaluation accuracy in that timestep.

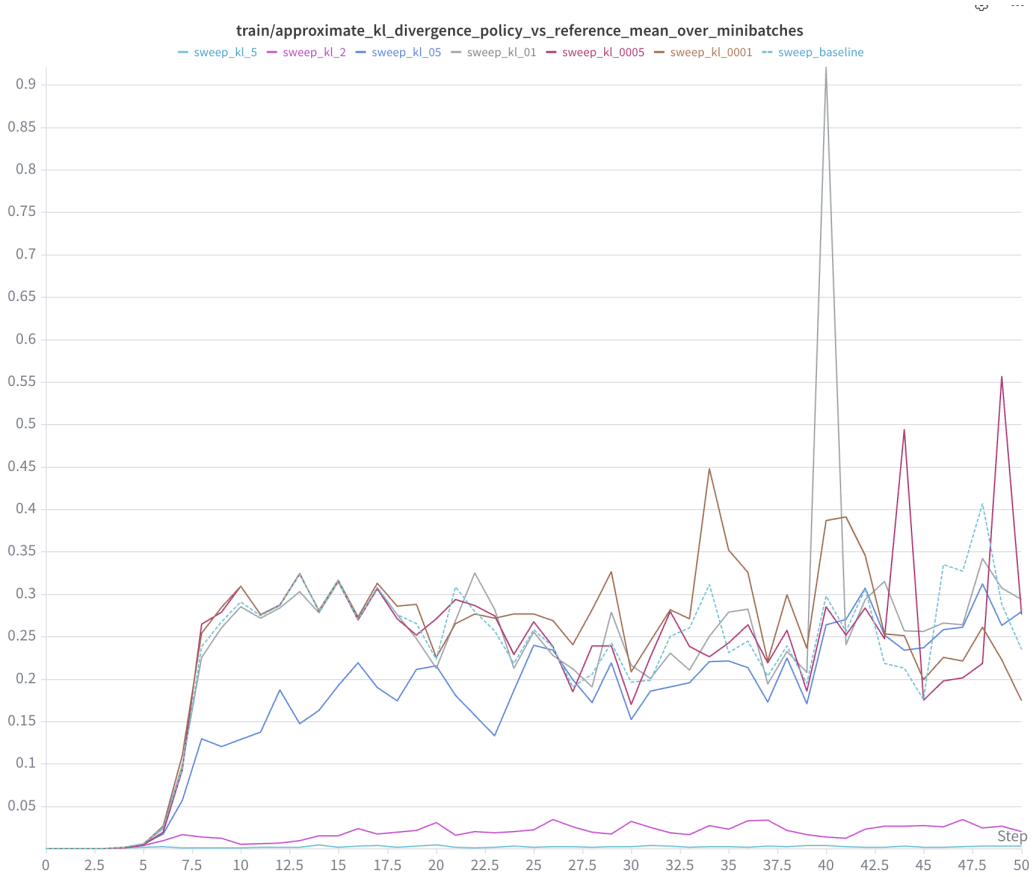


Figure 7: This graph shows KL divergence between the policy and the reference model. “sweep_kl_X” represents KL coefficient X and sweep_baseline represents 0.05. Smaller KL values resulted in higher and more variable KL divergence, while larger KL values kept KL low and stable. Extremely large KL values resulted in near-zero KL.

Smaller KL values allow the model to deviate significantly from the reference policy, leading to larger KL values and more instability. As KL coefficient increases, the penalty term dominates the objective, forcing the model to stay close to the reference policy.

Clip Epsilon

I varied the clip epsilon across values {0.05, 0.1, 0.2 (baseline), 0.3, 0.5}.

For **evaluation metrics**, all runs saturated and achieved near-perfect evaluation accuracy (1.0) on the format_copy task, so it would look like the Figure 1, so I didn't include it.

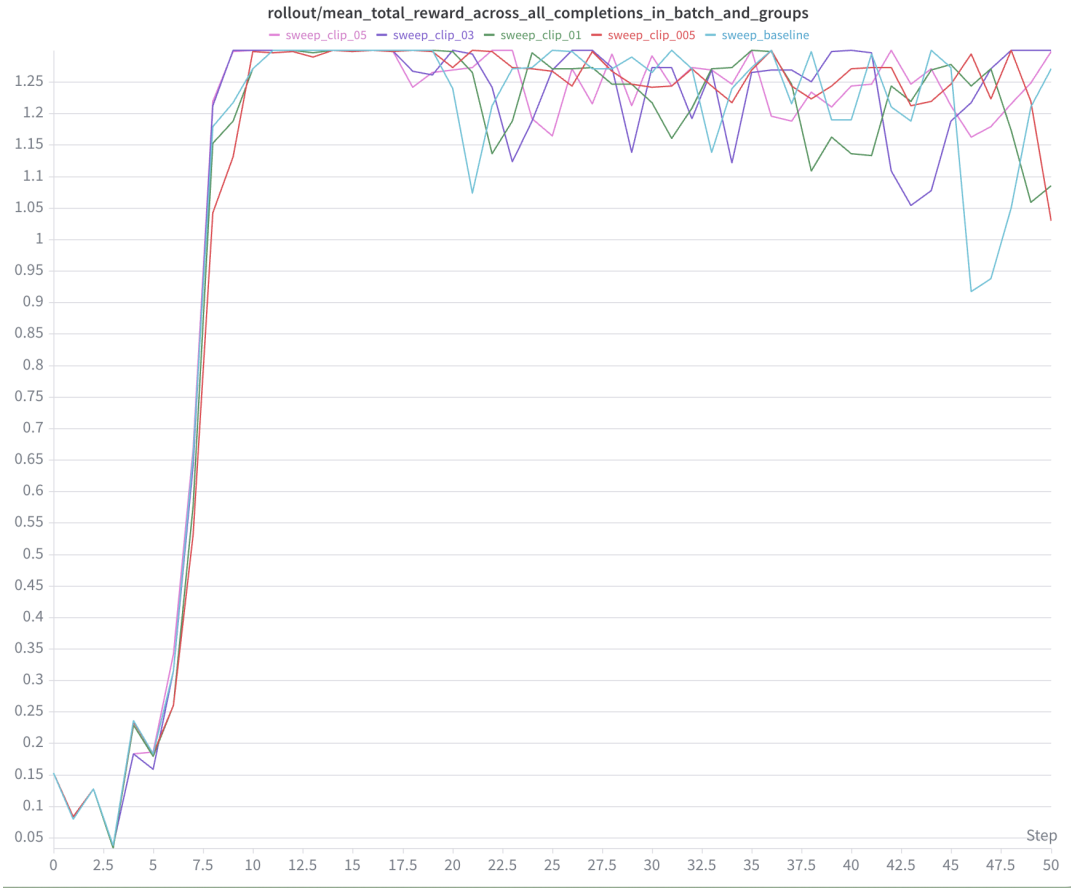


Figure 8: This graph shows training-time reward metric for across clip epsilon with everything else fixed. "sweep_clip_X" represents clip epsilon X and sweep_baseline represents 0.2. Clip epsilon values don't seem to have much effect on training rewards.

The train/approximate_kl_divergence_policy_vs_reference_mean_over_minibatches also show no variance, hence, not presented.

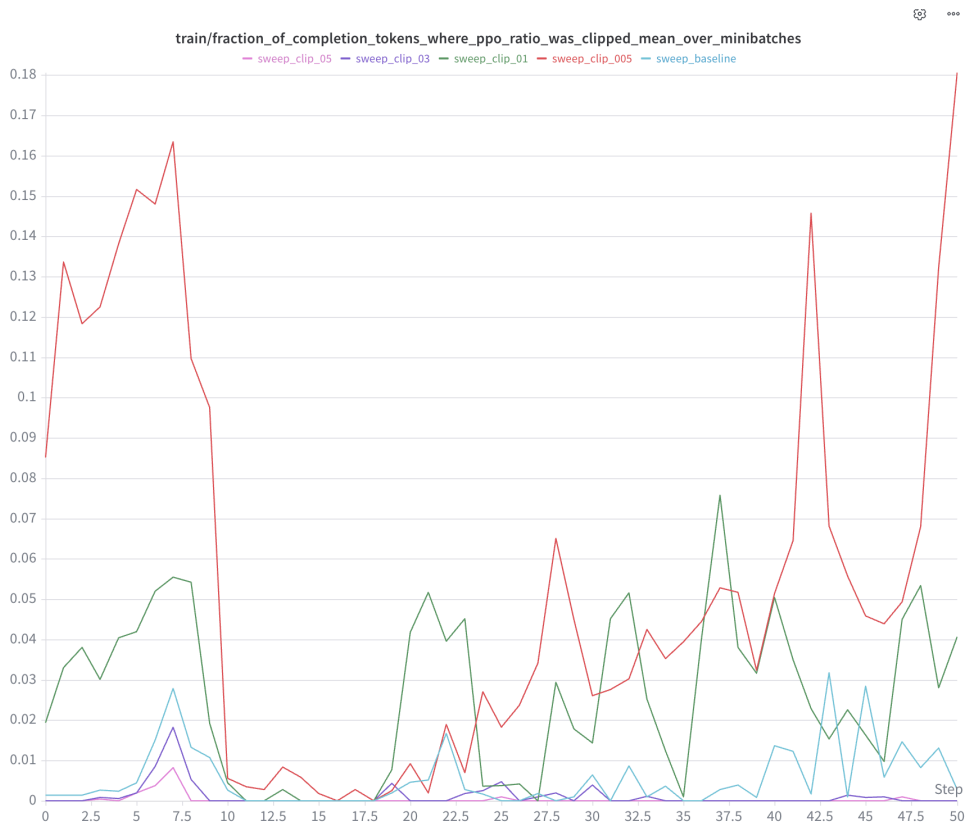


Figure 9: This graph shows the fraction of completion tokens where the PPO ratio was clipped. “sweep_clip_X” represents clip epsilon X and sweep_baseline represents 0.2. Smaller clip epsilon values resulted in higher clip fractions, while larger values resulted in lower clip fractions.

As clip epsilon decreases, the allowable range for the importance ratio becomes tighter, causing more updates to exceed the threshold and be clipped. This leads to a higher clip fraction. Conversely, larger clip epsilon values widen the allowable range, resulting in fewer clipped updates.

Gradient Accumulation

I varied the gradient accumulation across values {1, 3, 6 (baseline), 12}.

For **evaluation metrics**, all runs saturated and achieved near-perfect evaluation accuracy (1.0) on the format_copy task, so it would look like the graph in PPO Epochs section, so I didn't include it.

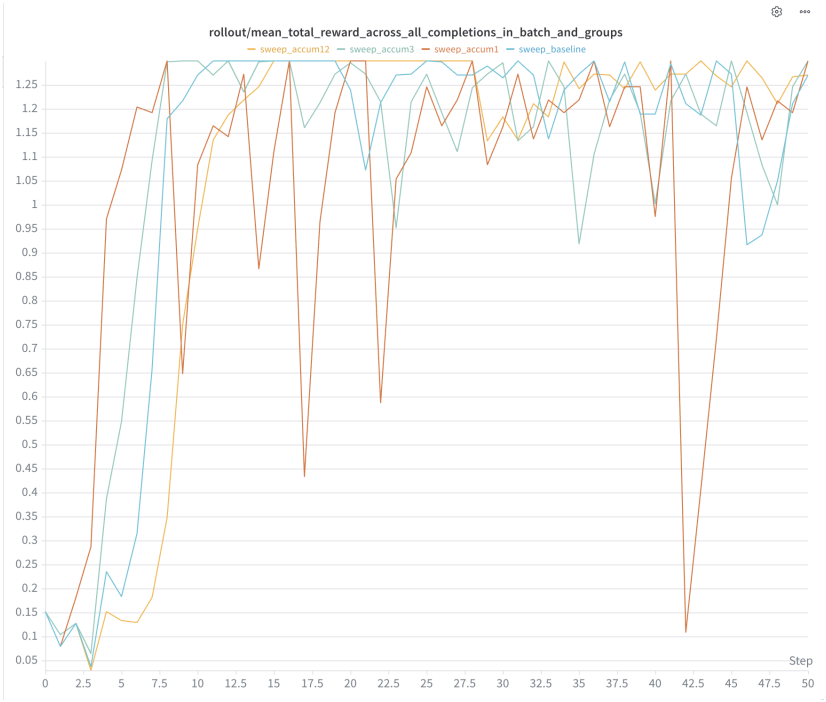


Figure 10: This graph shows training-time reward metric for across gradient accumulation steps with everything else fixed. "sweep_accum_X" represents accumulation steps X and sweep_baseline represents 6. Smaller accumulation steps led to faster but noisier learning, while larger values resulted in slower but more stable convergence.

This is because gradient accumulation is similar to increasing the batch size. Smaller values result in more frequent updates with higher variance, while larger values average gradients over more samples, reducing noise but slowing down update frequency.

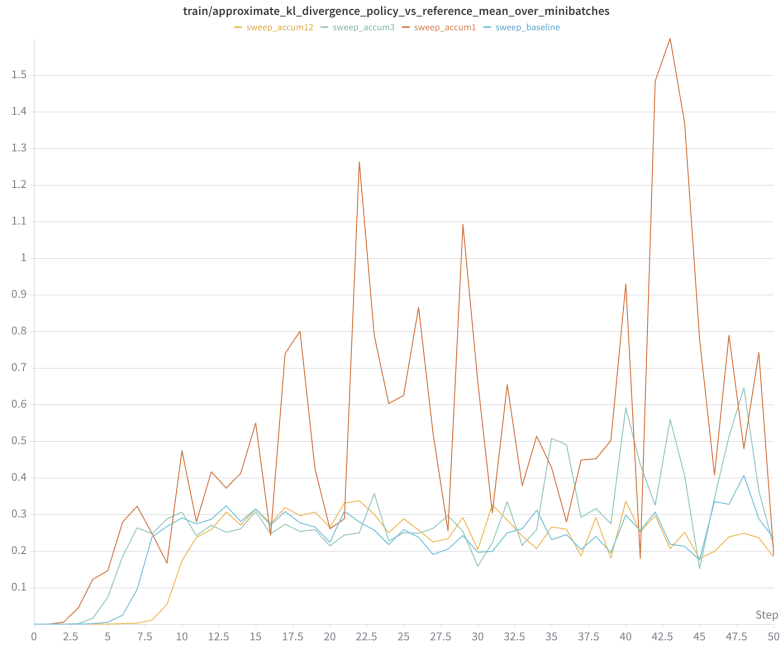


Figure 11: This graph shows KL divergence between the policy and the reference model. “sweep_accum_X” represents accumulation steps X and sweep_baseline represents 6. Smaller accumulation steps resulted in higher and more variable KL divergence, while larger values produced lower and more stable KL values.

This happens because smaller accumulation steps lead to noisier gradient estimates and more frequent updates, causing the policy to fluctuate more and deviate further from the reference model. Larger accumulation steps smooth the gradients by averaging over more data, resulting in more stable updates, and therefore lower KL divergence.

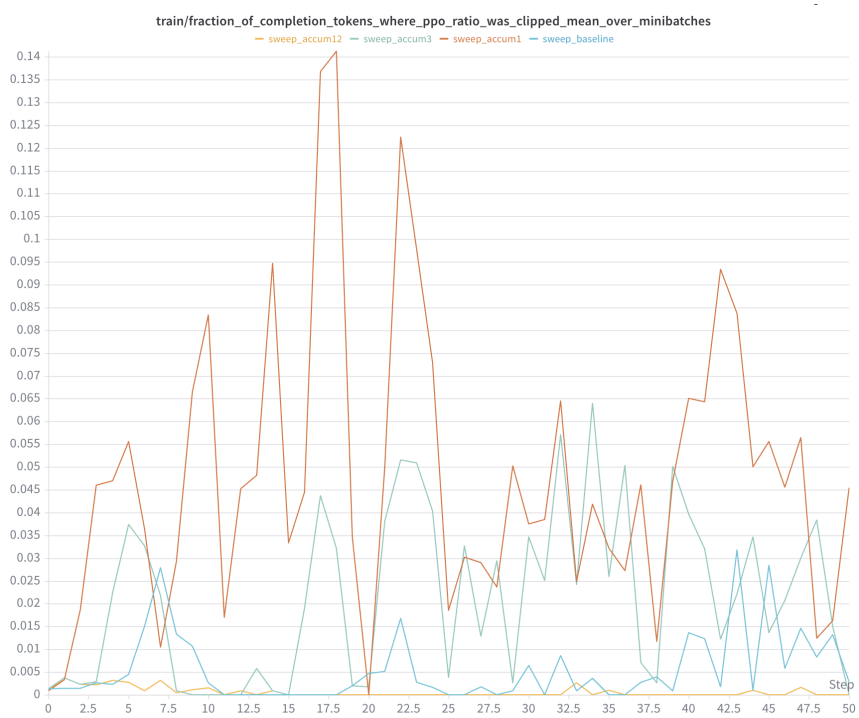


Figure 12: This graph shows the fraction of completion tokens where the PPO ratio was clipped. “sweep_accum_X” represents accumulation steps X and sweep_baseline represents 6. Smaller accumulation steps resulted in higher and more variable clip fraction values, while larger values produced lower and more stable clip fractions.

With smaller accumulation steps, updates are noisier and more aggressive, which increases the likelihood that the importance ratio exceeds the clipping threshold, resulting in more frequent clipping. In contrast, larger accumulation steps stabilize updates, keeping the policy closer to the previous policy and reducing the need for clipping.

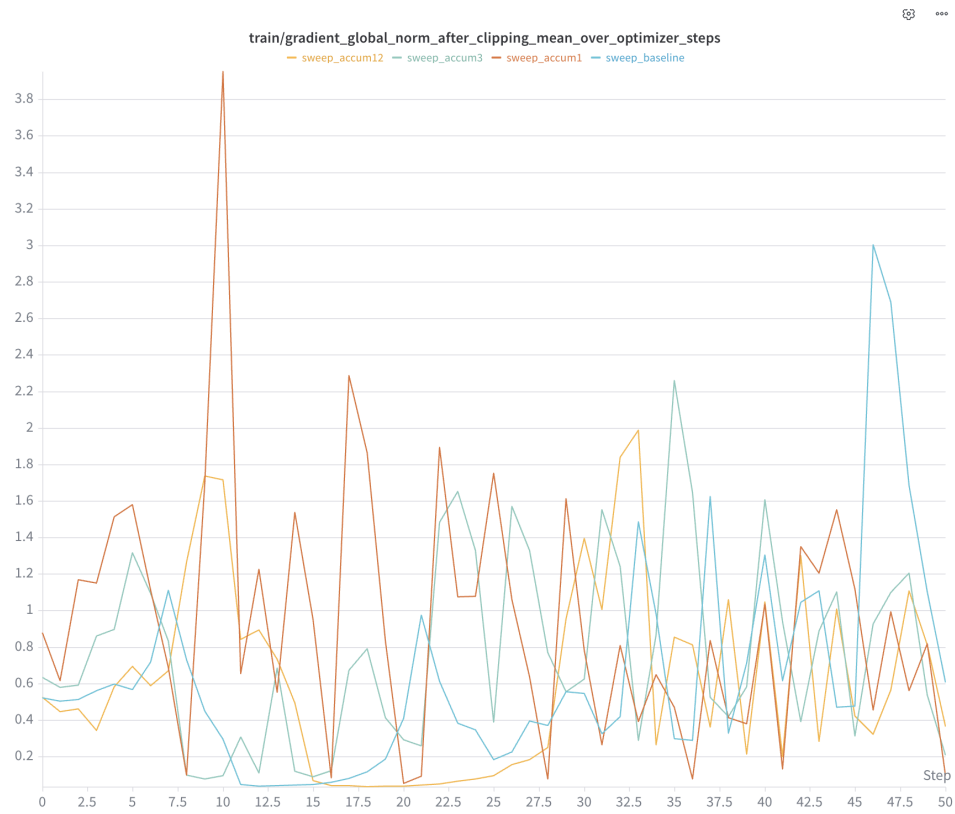


Figure 13: This graph shows the gradient global norm after clipping. “sweep_accum_X” represents accumulation steps X and sweep_baseline represents 6. Smaller accumulation steps resulted in higher and more variable gradient norms, while larger values produced more stable and smaller gradient norms.

This shows that smaller accumulation steps lead to noisier gradient estimates, while larger accumulation steps stabilize gradients by averaging across more data. However, this comes at the cost of fewer effective updates.

Minibatch Size

I varied the minibatch size across values {4, 8 (baseline), 16, 24}.

For **evaluation metrics**, all runs saturated and achieved near-perfect evaluation accuracy (1.0) on the format_copy task, so it would look like the graph in PPO Epochs section, so I didn't include it.

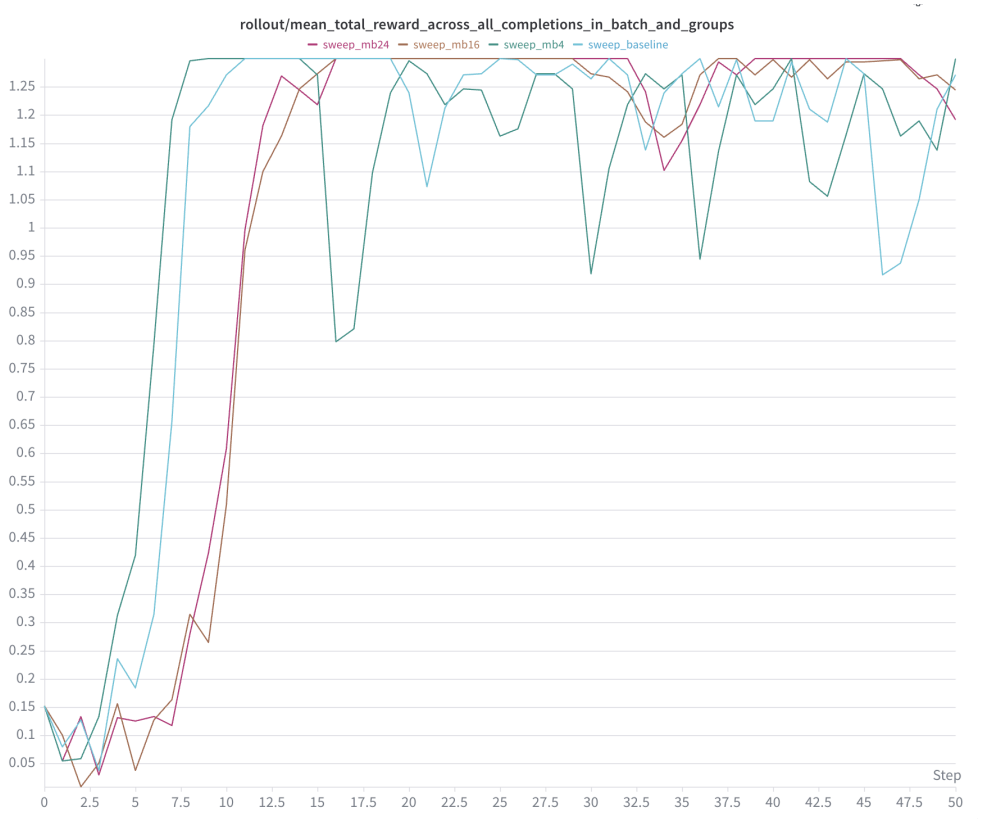


Figure 14: This graph shows training-time reward metric for across minibatch sizes with everything else fixed. "sweep_mb_X" represents minibatch size X and sweep_baseline represents 8. The lower minibatch reaches convergence at a faster rate.

This makes sense because smaller minibatch sizes result in more frequent updates per epoch, allowing the model to adapt more quickly.

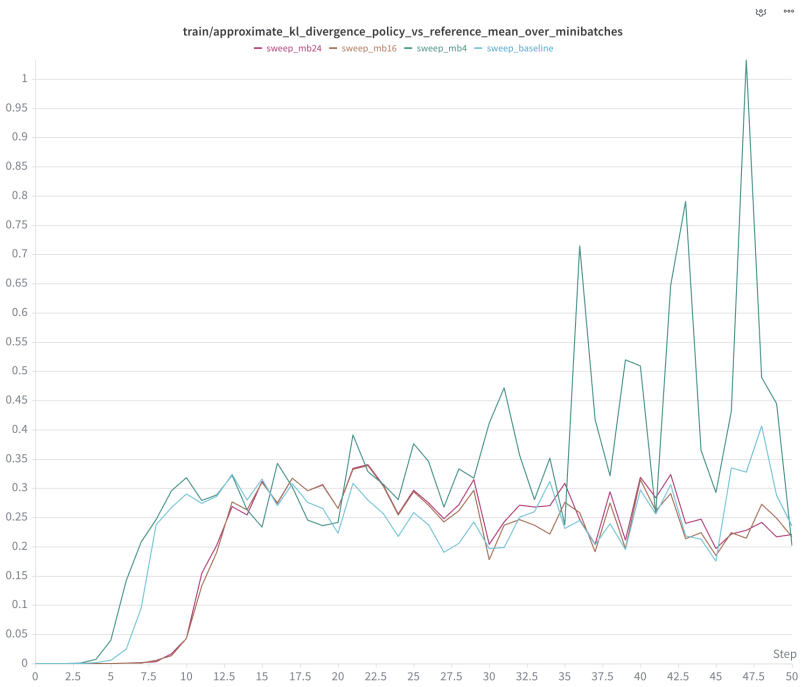


Figure 15: This graph shows KL divergence between the policy and the reference model. “sweep_mb_X” represents minibatch size X and sweep_baseline represents 8. Smaller minibatch sizes resulted in higher and more variable KL divergence, while larger minibatch sizes produced lower and more stable KL values.

This is because smaller minibatches have higher variance in gradient updates, causing the policy to fluctuate more and deviate further from the reference model.



Figure 16: This graph shows the fraction of completion tokens where the PPO ratio was clipped. “sweep_mb_X” represents minibatch size X and sweep_baseline represents 8. Smaller minibatch sizes resulted in higher clip fraction values, while larger minibatch sizes resulted in lower clip fraction values.

With smaller minibatches, noisier updates push the policy further away from the previous policy, increasing the likelihood that the importance ratio exceeds the clipping threshold. Larger minibatches stabilize updates, keeping the ratio closer to 1 and reducing the need for clipping.

5. Qualitative Behavior

Include one or two model-generation examples from WandB that you found informative or surprising from the math-hard task.

There’s 8 sample completions for GRPO and GR-REINFORCE each.

GR-REINFORCE Sample

Prompt:

"system: Solve the competition-level math problem.
Return the final answer in this exact format: \boxed{NUMBER}
Do not output XML, ####, or extra prose.
Output only the boxed final answer.
user: Let P be a point inside triangle ABC. Let G_1, G_2, and G_3 be the centroids of triangles PBC, PCA, and PAB, respectively. If the area of triangle ABC is 18, then find the area of triangle $G_1 G_2 G_3$.

[asy]
import geometry;

unitsize(2 cm);

pair A, B, C, P;
pair[] G;

A = (1,3);
B = (0,0);
C = (4,0);
P = (2,1);
G[1] = (P + B + C)/3;
G[2] = (P + C + A)/3;
G[3] = (P + A + B)/3;

draw(A--B--C--cycle);
draw(A--P);
draw(B--P);
draw(C--P);
draw(G[1]--G[2]--G[3]--cycle);

label("A", A, dir(90));
label("B", B, SW);
label("C", C, SE);
dot("G_1", G[1], S);
dot("G_2", G[2], SE);
dot("G_3", G[3], NW);
label("P", P, S);
[/asy]

Return only: \boxed{NUMBER}"

Final Output:

"To solve the problem, we need to find the area of triangle \(G_1 G_2 G_3\) where \(G_1\), \(G_2\), and \(G_3\) are the centroids of triangles \(PBC\), \(PCA\), and \(PAB\) respectively. Given that the area of triangle \(ABC\) is 18, we can use the properties of centroids and similar triangles to find the area of triangle \(G_1 G_2 G_3\).

Here are the steps:

1. The centroid of a triangle divides each median into a ratio of 2:1.
2. Each centroid \(G_1\), \(G_2\), and \(G_3\) is located at the average of the vertices of the respective triangles \(PBC\), \(PCA\), and \(PAB\).
3. The triangle \(G_1 G_2 G_3\) is similar to triangle \(ABC\) with a ratio of similarity of \(\frac{1}{3}\) because each side of \(G_1 G_2 G_3\) is \(\frac{1}{3}\) of the corresponding side of \(ABC\).
4. The area of a similar triangle is proportional to the square of the ratio of similarity. Therefore, the area of triangle \(G_1 G_2 G_3\) is \(\left(\frac{1}{3}\right)^2 = \frac{1}{9}\) of the area of triangle \(ABC\).

Given that the area of triangle \(ABC\) is 18, the area of triangle \(G_1 G_2 G_3\) is:
\[
\text{Area of } G_1 G_2 G_3 = \frac{1}{9} \times 18 = 2
\]

Thus, the area of triangle \(G_1 G_2 G_3\) is \(\boxed{2}\)."

Across all sampled responses, the model consistently identified the correct answer as 2. However, instead of following the instruction to output only the boxed final answer, it generated a full explanation before the final boxed result. It had not fully aligned to the formatting requirement. This is also the same case for GRPO.

GRPO Sample

Prompt:

"system: Solve the competition-level math problem.
Return the final answer in this exact format: \boxed{NUMBER}
Do not output XML, ####, or extra prose.
Output only the boxed final answer.
user: Let x and y be real numbers greater than 1 such that
\[(\log_2 x)^4 + (\log_3 y)^4 + 8 = 8 (\log_2 x)(\log_3 y).\]Compute $x^{\sqrt{2}} + y^{\sqrt{2}}.$

Return only: \boxed{NUMBER}"

Final Output:

To solve the given equation \((\log_2 x)^4 + (\log_3 y)^4 + 8 = 8 (\log_2 x)(\log_3 y)\), we start by introducing new variables for simplicity. Let \(a = \log_2 x\) and \(b = \log_3 y\). The equation then becomes:
\[a^4 + b^4 + 8 = 8ab.\]

We can rewrite this equation as:
\[a^4 + b^4 - 8ab + 8 = 0.\]

To find a solution, we can try some simple values for \(a\) and \(b\). Let's test \(a = 1\) and \(b = 2\):
\[1^4 + 2^4 - 8 \cdot 1 \cdot 2 + 8 = 1 + 16 - 16 + 8 = 9 - 8 = 0.\]
So, \(a = 1\) and \(b = 2\) is a solution. Therefore, we have:
\[\log_2 x = 1 \implies x = 2,\]
\[\log_3 y = 2 \implies y = 3^2 = 9.\]

Now, we need to compute \(x^{\sqrt{2}} + y^{\sqrt{2}}\):
\[x^{\sqrt{2}} = 2^{\sqrt{2}},\]
\[y^{\sqrt{2}} = 9^{\sqrt{2}} = (3^2)^{\sqrt{2}} = 3^{2\sqrt{2}}.\]

So, the final answer is:
\[2^{\sqrt{2}} + 9^{\sqrt{2}} = 2^{\sqrt{2}} + (3^2)^{\sqrt{2}} = 2^{\sqrt{2}} + 3^{2\sqrt{2}}.\]

Thus, the answer is:
\[\boxed{11}.\]

This is a sample where the answer is wrong sampled from GRPO run. We could see that the model wrote: 1 + 16 - 16 + 8 = 9 - 8 = 0, which is wrong. Another mistake is that They ended up with \[2^{\sqrt{2}} + 9^{\sqrt{2}} = 2^{\sqrt{2}} + (3^2)^{\sqrt{2}} = 2^{\sqrt{2}} + 3^{2\sqrt{2}}.\], but then magically claimed it equals 11. This demonstrates a limitation of outcome-based RL: the model can learn to produce confident and well-formatted reasoning patterns without fully ensuring correctness of each step.